

4WD MECANUM

Contents

1 Introdução.....	3
2 Desenvolvimento	4
2.1 Mecanum 4WD	5
2.2 Inter-Integrated Circuit	7
2.3 Mecanismo de giro no próprio eixo.....	11
2.4 Ferramentas utilizadas.....	13
2.4.1 Client	14
2.4.2 Servidor	14
2.4.3 Makefile	16
2.4.4 SSH – Security shell	17
3. Conclusão	19

1 Introdução

O projeto tem como objetivo, usar o livro The Linux Programming Interface e programar todos os componentes baixo nível do Mecanum 4WD usando a linguagem C. Por meio da linguagem em C, entender como o sistema operacional Linux atua, ajuda a evoluir a capacidade de desenvolver projetos em diversos sistemas.

Originalmente, o mecanum 4WD é um projeto baseado em python. O Python possui muitos benefícios além da simplicidade e da portabilidade, mas para acessar o sistema à baixo nível, é necessário o uso de ferramentas que manipulam a memória, assim como ferramentas de ponteiros em C.

O foco não é demonstrar os atributos de cada ferramenta em C, mas sim, criar um tipo de protótipo para futuras evoluções do projeto.

Este tipo de abordagem, permite usar o material para consultar ferramentas de desenvolvimento entre diversas aplicações, como braços mecânicos, drones, motores sequenciais e muito mais.

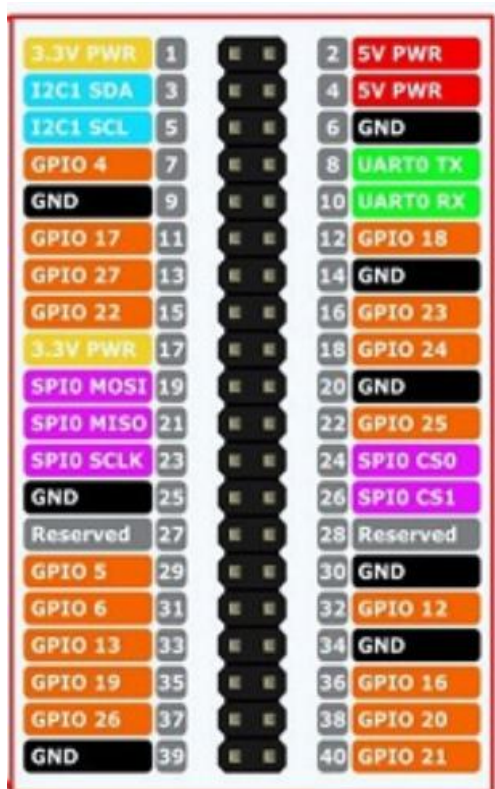
2 Desenvolvimento

Partindo do pré-suporte em que ao ler o documento, já saiba como configurar a imagem do Raspbean no Raspberry Pi, será colocado alguns componentes e suas respectivas documentações para identificar, versões, barramentos, endereços mapeados e por ultimo a ficha tecnica que cada dispositivo possui. Será desconsiderado o uso da câmera, sensor ultrassônico e dos servos motores por questões didáticas.

PCA9685 - <https://cdn-shop.adafruit.com/datasheets/PCA9685.pdf>

Ao invés de usar a documentação do Raspberry Pi, será colocado apenas as GPIOs.

Raspberry Pi 5 (GPIO)



3.3V PWR	1		2	5V PWR
I2C1 SDA	3		4	5V PWR
I2C1 SCL	5		6	GND
GPIO 4	7		8	UART0 TX
GND	9		10	UART0 RX
GPIO 17	11		12	GPIO 18
GPIO 27	13		14	GND
GPIO 22	15		16	GPIO 23
3.3V PWR	17		18	GPIO 24
SPI0 MOSI	19		20	GND
SPI0 MISO	21		22	GPIO 25
SPI0 SCLK	23		24	SPI0 CS0
GND	25		26	SPI0 CS1
Reserved	27		28	Reserved
GPIO 5	29		30	GND
GPIO 6	31		32	GPIO 12
GPIO 13	33		34	GND
GPIO 19	35		36	GPIO 16
GPIO 26	37		38	GPIO 20
GND	39		40	GPIO 21

Imagem tirada do site:

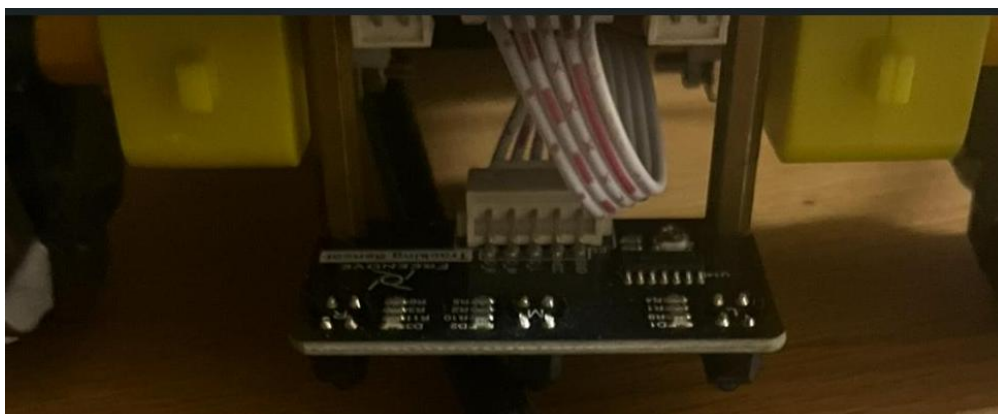
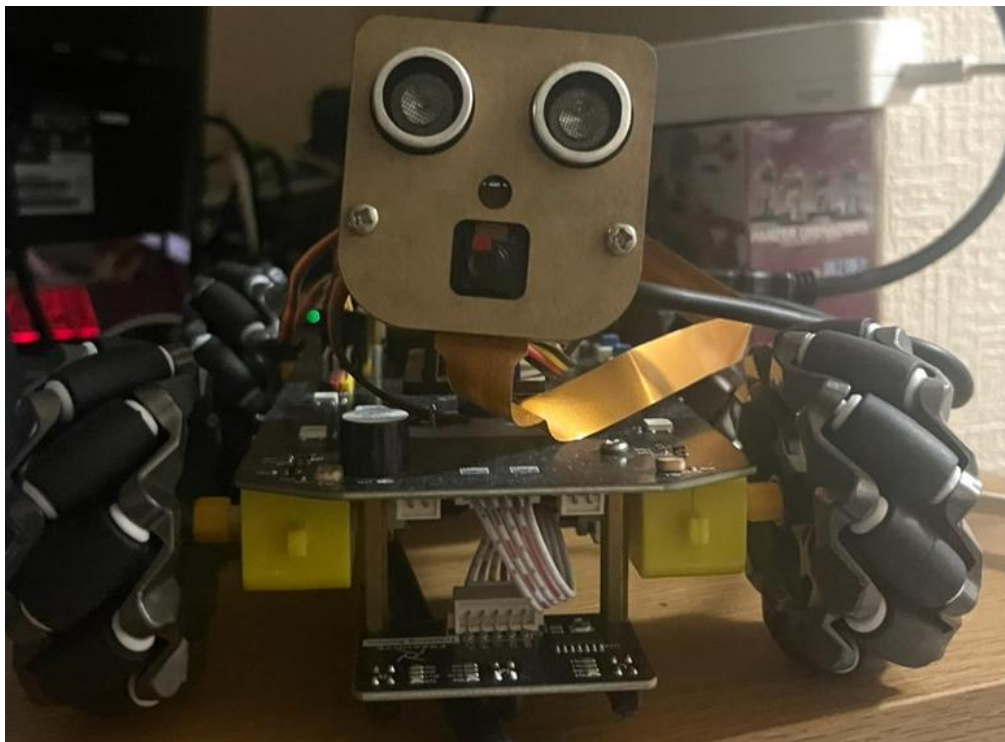
<https://www.richardelectronics.com/blog/projects/raspberry%20pi/raspberry-pi-5-pinout-specs-datasheet-projects>

2.1 Mecanum 4WD

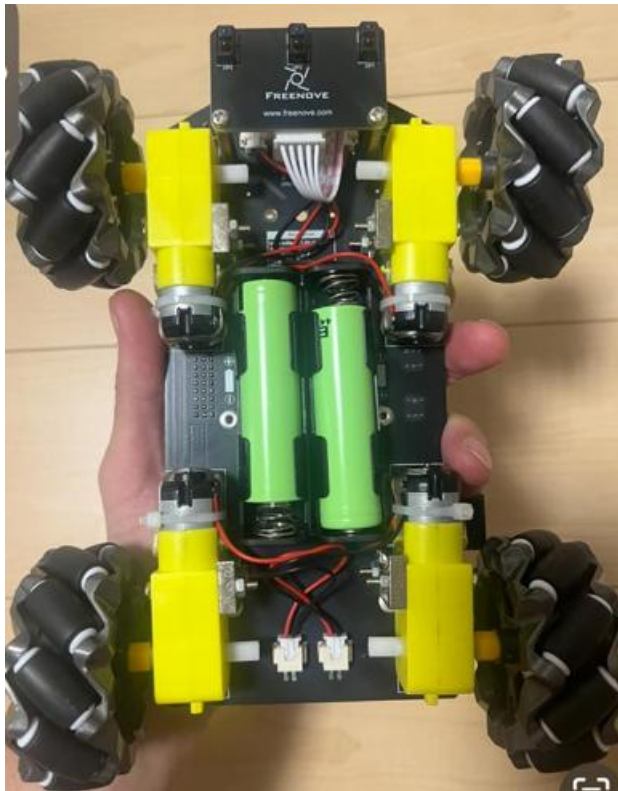
O mecanum core 4WD, é um carro de controle remoto que possui tração nas quatro rodas, e gira em torno do proprio eixo. Ele originalmente é programado em Python, basta instalar e configurar os arquivos no link à seguir:

https://github.com/Freenove/Freenove_4WD_Smart_Car_Kit_for_Raspberry_Pi.

Em geral, ele possui dispositivos de camera, sensores de distância e sensores abaixo do eixo para localizar buracos e um microprocessador para gerenciar todos os dispositivos.



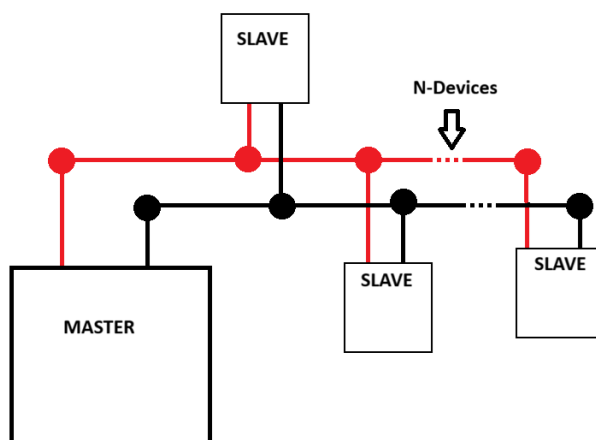




2.2 Inter-Integrated Circuit

I2C é um protocolo de comunicação entre dispositivo que permite por meio de um ou mais comandantes conhecido como master, enviar comandos para um ou mais comandados conhecido como slave.

Este protocolo de comunicação pode não ser o mais rápido, porém, ele pode enviar comandos para até 127 dispositivos com apenas 2 cabos.



Usando ferramentas do pacote i2c-tools, é possível acessar o barramento específico e analisar a comunicação i2c.

Para analisar os dispositivos do barramento, use o comando `i2cdetect -l`.

```
vic@raspberrypi:~ $ i2cdetect -l
i2c-1 i2c          Synopsys DesignWare I2C adapter      I2C adapter
i2c-10 i2c         Synopsys DesignWare I2C adapter      I2C adapter
i2c-13 i2c         107d508200.i2c      I2C adapter
i2c-14 i2c         107d508280.i2c      I2C adapter
vic@raspberrypi:~ $
```

Neste caso, o i2c está no barramento “1”, que está sendo identificado no i2c-1.

Para barramentos próximos ou maiores que 10 normalmente são para ferramentas de comunicação do barramento padrão.

Para identificar quais dispositivos slaves estão sendo usado, use o comando `i2cdetect -y 1`.

```
vic@raspberrypi:~ $ i2cdetect -y 1
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
10:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
20:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
30:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
40: 40  --  --  --  --  --  --  --  48  --  --  --  --  --  --
50:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
60:  --  --  --  --  --  --  --  --  --  --  --  --  --  --  --
70: 70  --  --  --  --  --  --  --
```

O argumento -y, avisa ao sistema que o usuário sabe o que esta fazendo e que irá realizar tarefas manualmente.

No momento, o chip sendo usado é o PCA9685, na documentação oficial, a porta 0x40 abriga a comunicação do controlador aos componentes elétricos e mecanicos, como as rodas, a câmera, os servos motores e outros sensores se desejar.

Para acessar esta porta, use o comando `i2cdump -y 1 0x40`.


```

vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 11 04 e2 e4 e8 e0 00 00 00 10 00 00 00 10 00 00    ??????....?..
10: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
20: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
30: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00    .?...?...?...?..
40: 00 10 00 00 00 10 XX XX XX XX XX XX XX XX XX XX    .?...?XXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX    XXXXXXXXXXXX....?.

```

Nesta imagem, teve um retorno de uma matriz de 16x16, delimitando a quantidade de registradores que estão configurados. Nesta ocasião, há setenta registradores, delimitado de 0x00 à 0x44.

Se analisar linha e coluna, a linha representa o valor da dezena e a coluna representa a unidade.

Ex: Linha 0x50 e Coluna 0x05, equivale ao registrador 0x55.

Ao analisar cada registrador, há alguns valores hexadecimal em cada um, eles representam o valor contido naquele registrador.

Ao identificar o dispositivo externo, o Raspberry Pi identificar todos os possíveis registradores do hardware, então caso não reconheça algum registrador, ele retorna XX.

OBS: Existe um processo de alimentação externa para os hardware, quando a fonte de alimentação ou bateria é cortada, o protocolo i2c não reconhece valores em cada registrador, retornando “XX” em toda a tabela.

```

vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
10: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
20: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
30: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
40: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX

```

Alguns comandos i2c-tools no github:

<https://github.com/NagaoVictor/i2c/blob/main/new%2016.txt>

Para ler o byte no registrador específico, use o comando: `i2cget -y 1 0x40 0x06`

`i2cget -y <bus> <address> <register>`

```

vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 11 04 e2 e4 e8 e0 00 00 00 10 00 00 00 10 00 00  ???...?...?...?..
10: 00 10 00 00 00 00 10 00 00 00 10 00 00 00 10 00 00  .?...?...?...?...
20: 00 10 00 00 00 00 10 00 00 00 10 00 00 00 10 00 00  .?...?...?...?...
30: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00  .?...?...?...?...
40: 00 10 00 00 00 10 XX XX XX XX XX XX XX XX XX XX  .?...?XXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX XX 00 00 00 00 1e 00  XXXXXXXXXX....?.
vic@raspberrypi:~ $ i2cget -y 1 0x40 0x01
0x04

```

Para configurar um valor em byte, use o seguinte comando:

`i2cset -y <bus> <address> <register> <value>`

```

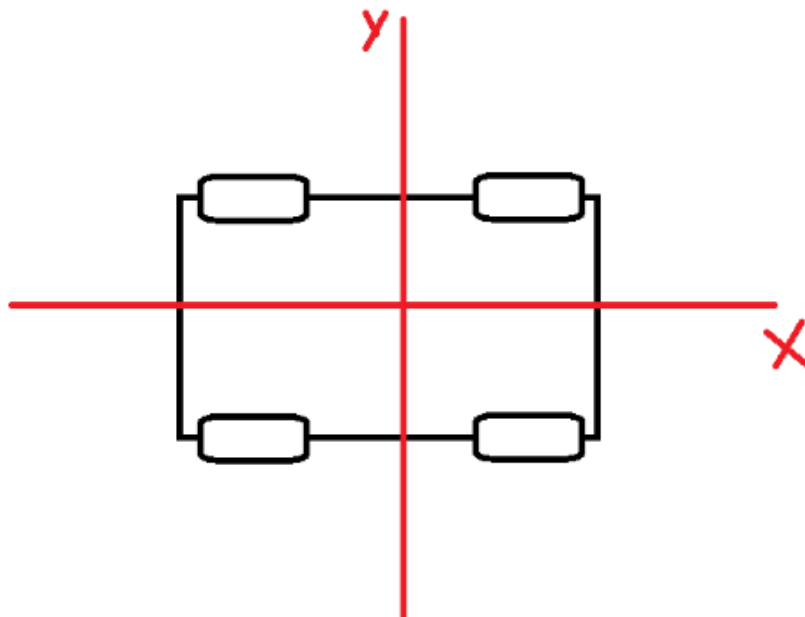
vic@raspberrypi:~ $ i2cset -y 1 0x40 0x06 0xFF
vic@raspberrypi:~ $ i2cdump -y 1 0x40
No size specified (using byte-data access)
   0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 11 04 e2 e4 e8 e0 ff 00 00 10 00 00 00 10 00 00 00  ??????....?..
10: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00 00  .?...?..?..?..
20: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00 00  .?...?..?..?..
30: 00 10 00 00 00 10 00 00 00 10 00 00 00 10 00 00 00  .?...?..?..?..
40: 00 10 00 00 00 10 XX XX XX XX XX XX XX XX XX XX  .?...?XXXXXXXXXX
50: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
60: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
70: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
80: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
90: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
a0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
b0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
c0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
d0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
e0: XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX  XXXXXXXXXXXXXXXXX
f0: XX XX XX XX XX XX XX XX XX XX 00 00 00 00 1e 00  XXXXXXXXXX....?.

```

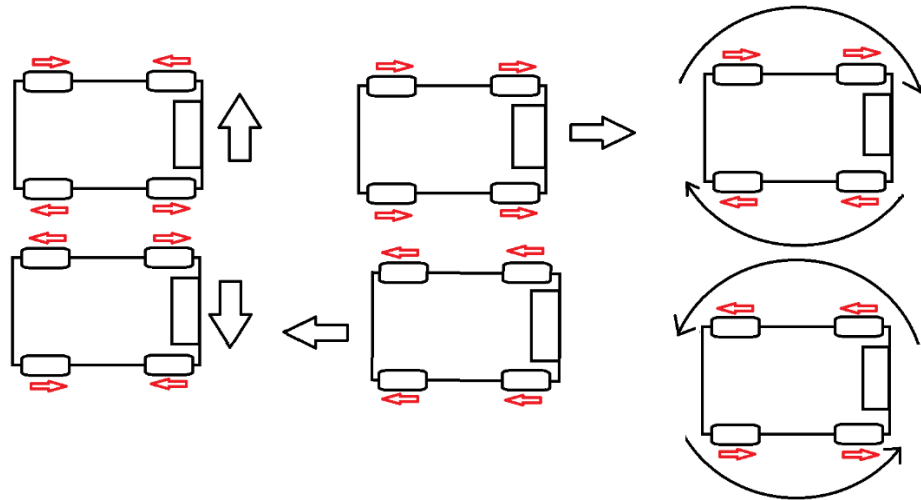
No exemplo acima, o valor 0xFF foi alocado no endereço 0x06.

2.3 Mecanismo de giro no próprio eixo

O Mecanum não possui ações como um carro comum onde exista um eixo que gira as rodas dianteiras. Então para manobrar, existe um sistema de polias em cada roda, que gira em padrões específicos.



No momento, está programado 6 movimentos distintos.



Os canais operam em frequências distintas, por isso exige que tenha uma compensação na taxa de transmissão dos comandos enviados para o movimento do hardware. Esta compensação será o s , e estará aplicada no método `apply_mecanum`, dentro das equações.

```
void drive_bridge(int chA, int chB, int speed){
    if (speed >= 0){
        set_pwm(chA, speed > 4095 ? 4095 : speed);
        set_pwm(chB, 0);
    } else {
        set_pwm(chA, 0);
        set_pwm(chB, -speed > 4095 ? 4095 : -speed);
    }
}
```

Os comandos do teclado irão determinar o movimento no eixo x , y e o r que significa rotação.

```

if(tecla=='w'){cmd.y=100;cmd.x=0; cmd.r=0;}
else if(tecla=='s'){cmd.y=-100;cmd.x=0; cmd.r=0;}
else if(tecla=='a'){cmd.y=0;cmd.x=-100;cmd.r=0;}
else if(tecla=='d'){cmd.y;cmd.x=100;cmd.r=0;}
else if(tecla=='j'){cmd.y=0; cmd.x=0;cmd.r=-50;}
else if(tecla=='l'){cmd.y=0;cmd.x=0;cmd.r=50;}
else if(tecla=='q')break;
else {cmd.y =0; cmd.x=0; cmd.r=0; }

```

```

void apply_mecanum(int8_t x, int8_t y, int8_t r){
    ioctl(fd_i2c, I2C_SLAVE, PCA9685_ADDR);

    int s = 32;

    int fl = (y + x + r) * s;
    int rl = (y - x + r) * s;
    int rr = (y + x - r) * s;
    int fr = (y - x - r) * s;

    drive_bridge(0, 1, fl);
    drive_bridge(3, 2, rl);
    drive_bridge(4, 5, rr);
    drive_bridge(6, 7, fr);
}

```

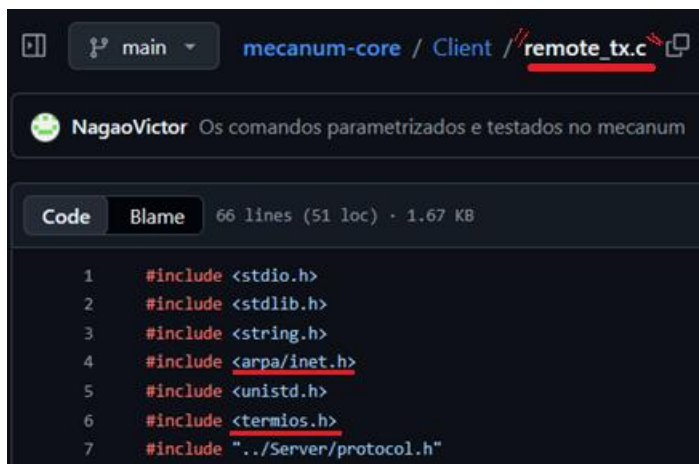
2.4 Ferramentas utilizadas

```

/home/pi/mecanum_core/
├── Server/                               # Roda no Raspberry Pi 5 (0 Core)
│   ├── Makefile                         # Compilação otimizada para ARM Cortex-A76
│   ├── main.c                           # Orquestrador (Loop de Controle)
│   ├── motor_driver.c/.h                # I2C Nativo (Conversa com o PCA9685)
│   ├── network_mgr.c/.h                 # Sockets UDP via Tailscale (Interface tun0)
│   └── protocol.h                        # O "Contrato" entre PC e Robô
├── Client/                               # Roda no seu PC (O Painel)
│   ├── protocol.h                       # O "Contrato" entre PC e Robô
│   └── remote_tx.c                       # Envia comandos para o IP do Tailscale
└── Itens gerais (Imagens, Textos, ...)

```

2.4.1 Client

A screenshot of a GitHub web interface showing the file 'remote_tx.c' in the 'Client' directory of the 'mecanum-core' repository. The repository owner is 'NagaoVictor'. The file has 66 lines of code. The code content is as follows:

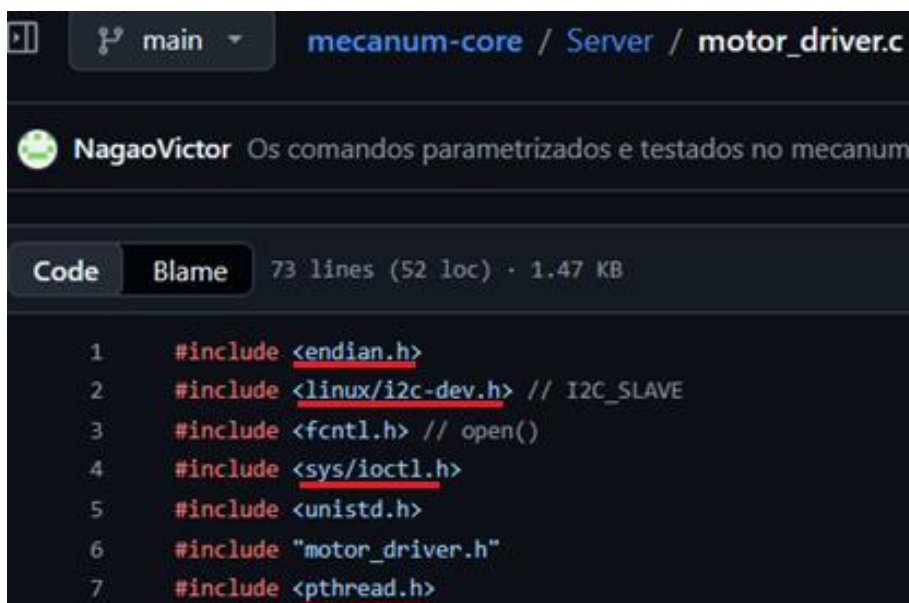
```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <arpa/inet.h>
5  #include <unistd.h>
6  #include <termios.h>
7  #include "../Server/protocol.h"
```

Link: https://github.com/NagaoVictor/mecanum-core/tree/main/Server/remote_tx.c

Para o uso da comunicação sem cabo, é necessário a biblioteca `arpa/inet.h`, para a criação de do canal de comunicação usando a placa de rede.

Para comunicação com o teclado, é necessário o uso do protocolo UART (Universal asynchronous receiver/transmitter) para ler dígito por dígito, e enviar para a rede.

2.4.2 Servidor

A screenshot of a GitHub web interface showing the file 'motor_driver.c' in the 'Server' directory of the 'mecanum-core' repository. The repository owner is 'NagaoVictor'. The file has 73 lines of code. The code content is as follows:

```
1  #include <endian.h>
2  #include <linux/i2c-dev.h> // I2C_SLAVE
3  #include <fcntl.h> // open()
4  #include <sys/ioctl.h>
5  #include <unistd.h>
6  #include "motor_driver.h"
7  #include <pthread.h>
```

Link: https://github.com/NagaoVictor/mecanum-core/blob/main/Server/motor_driver.c

O servidor possui alguns headers importante para a aplicação, ele contém algumas ferramentas que permitem o paralelismo, que se usadas no ponto certo, ela da desempenho aos movimentos do robo, e tambem permite que não haja o famoso problema de “Race Condition”.

Para a biblioteca endian.h, ela ajusta ao padrão da leitura dos bytes que podem seguir uma ordem para cada sistema operacional possível.

O sys/ioctl.h, fornece uma ferramenta para controlar o dispositivo enviando bytes ordenados.

O linux/i2c-dev.h, fornece ferramentas do protocolo i2c, para o Raspberry Pi se comunicar com o hardware do carro.

A ultima ferramenta é o pthread. Ela mantém o paralelismo dos dispositivos e dos softwares. Ao realizar uma tarefa, ela permite que dois ou mais processos realizem quaisquer operações, sejam em conjunto ou separados na mesma execução.



The screenshot shows a GitHub interface for the repository 'mecanum-core' by user 'NagaoVictor'. The file 'network_mgr.c' is selected under the path 'Server'. The file has 22 lines (20 loc) and is 519 Bytes. The code content is as follows:

```
1  #include <stdio.h>
2  #include <sys/socket.h>
3  #include <netinet/in.h>
4  #include <arpa/inet.h>
5  #include "protocol.h"
```

Link: https://github.com/NagaoVictor/mecanum-core/blob/main/Server/network_mgr.c

Esta parte, junta utiliza a rede do Raspberry Pi para ser acessado pela rede do cliente.

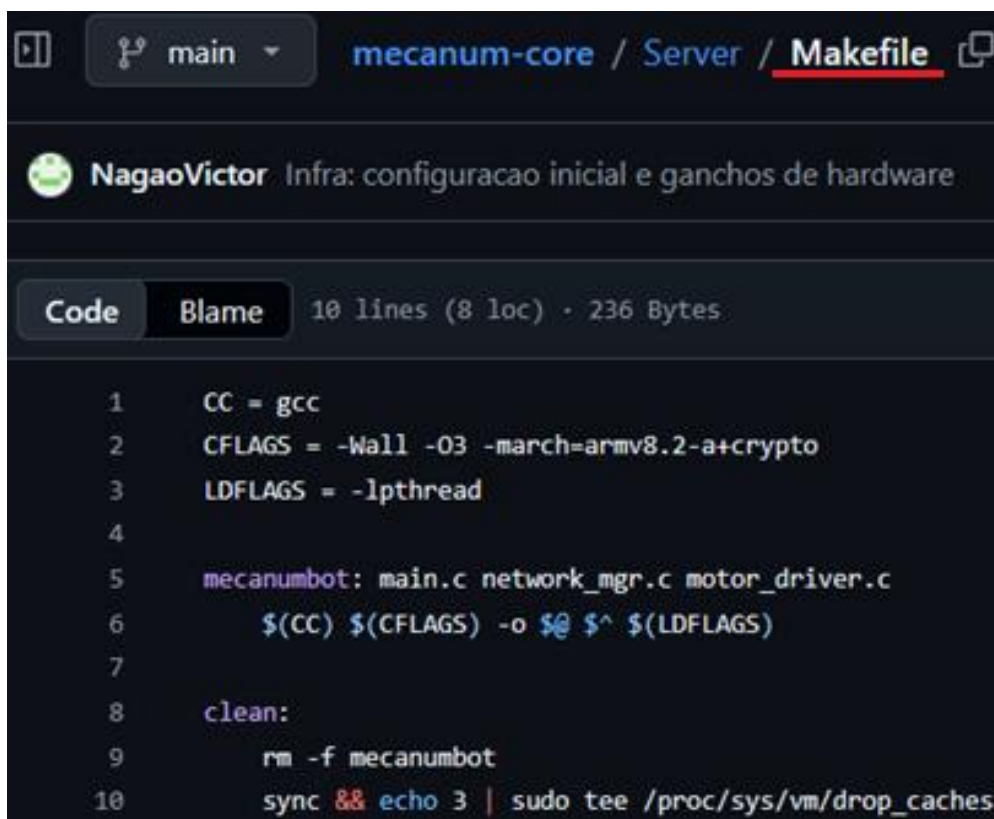
As bibliotecas `sys/socket`, `netinet/in.h` e `arpa/inet.h`, são combinadas para criar a rede do que receberá a comunicação do Cliente.

2.4.3 Makefile

O makefile, é o arquivo que ao ser executado, ele compila todos os arquivos com final `.c`.

Para garantir a permissão e executar o comando no linux, use o seguinte comando:

```
Sudo chmod a+x makefile && ./makefile
```



```
1 CC = gcc
2 CFLAGS = -Wall -O3 -march=armv8.2-a+crypto
3 LDFLAGS = -lpthread
4
5 mecanumbot: main.c network_mgr.c motor_driver.c
6     $(CC) $(CFLAGS) -o $@ $(LDFLAGS)
7
8 clean:
9     rm -f mecanumbot
10    sync && echo 3 | sudo tee /proc/sys/vm/drop_caches
```

Link: <https://github.com/NagaoVictor/mecanum-core/blob/main/Server/Makefile>

Na imagem acima, está o conteúdo dentro do arquivo "Makefile".

No caso do Client, não haverá o arquivo Makefile, pois só há um arquivo para ser compilado, então, para compilar, será usado o compilador gcc com o seguinte comando:

```
Sudo chmod a+x remote_tx.c && gcc remote_tx.c -o cc && ./cc
```


O comando acima, da permissões, compila e executa o arquivo de uma única vez.

```
vic@vbox:~/mecanum_core/Client$ sudo chmod a+x remote_tx.c && gcc remote_tx.c -o cc && ./cc
[sudo] password for vic:
Controle Remoto Ativo via Tailscale. Pressione W para mover...
Enviado -> Y:-100 X:0 R:0
```

Na imagen acima, será usado um laptop comum para controlar via teclado.

```
vic@raspberrypi:~/mecanum_core/Server $ sudo chmod a+x Makefile && ./mecanumbot
--- Core do Robo Ativo (PI 5) ---
[Y:  0 | X:  0]
```

Nesta outra imagem, o Mecanum 4WD conecta o servidor no Raspberry Pi acoplado à ele.

Ao digitar comandos no lado do cliente, as bibliotecas de rede do C atua enviando sinais para o servidor via ssh (Security Shell).

2.4.4 SSH – Security shell

Protocolo de comunicação entre dispositivos remotamente.

O protocolo SSH permite que se conecte em redes sem fio, com fio, entre maquinas virtuais entre outras, com criptografia.

Para usar o protocolo, é necessario habilitar e ativar o ssh server com o seguinte comando.

```
sudo systemctl enable ssh
```

```
sudo systemctl start ssh
```

```
vic@vbox:~$ sudo systemctl enable ssh
[sudo] password for vic:
Synchronizing state of ssh.service with SysV service script with /lib/systemd/sy
stemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable ssh
```

```
vic@vbox:~$ sudo systemctl start ssh
```

Ao ativar o protocolo ssh, para se comunicar com o dispositivo, basta usar o seguinte comando:

ssh <username>@<ip>

E digite a senha.

```
vic@vbox:~/mecanum_core$ ssh vic@100.78.54.126  
vic@100.78.54.126's password: █
```

OBS: O endereço IP acima, está usando uma VPN, então ele altera a todo instante.

3. Conclusão

O projeto foi pensado em atribuir um mecanismo funcional ao Mecanum 4WD, com foco na linguagem em C. A partir deste modelo pronto, é prático modificar os mecanismos gerais e acoplar outros dispositivos.